TGDetect — Heterogeneous Continuous-Time TGNN Architecture

In-Depth Architecture Study · Multi-Step Cyberattack Detection Framework
Aligned with Objectives O1–O4

LAYER 1 — MULTI-SOURCE DATA INGESTION & PREPROCESSING

OBJ 1

1.1 Heterogeneous Log Source Collection

- Authentication Logs
- System Audit Logs
- Network Flow Data
- DNS Query Logs
- Cloud API Logs

Entity Types: Users, Processes, Files, IP Addresses, Domains, Ports, Cloud Resources, Registry Keys

Event Types: Process Creation, File Read/Write, Network Connection, DNS Resolution, Authentication, Privilege Escalation, API Call

1.2 Temporal Normalization & Entity Resolution

NORMALIZATION PIPELINE

Raw Timestamps → UTC Standardization → Continuous-Time → $t \in \mathbb{R}^+$

Entity Resolution: Deduplication via hashing (SHA-256 of entity attributes). Cross-source identity linking (e.g., PID-to-User mapping, IP-to-Domain correlation).

SCHEMA UNIFICATION

Unified Event Schema
(src, dst, action, t, attrs)

Feature Extraction: Categorical encoding of action types, numerical fea from event attributes, temporal features from timestamp deltas.

LAYER 2 — HETEROGENEOUS TEMPORAL GRAPH CONSTRUCTION

OBJ 1

2.1 Graph Schema Definition

Nodes (Entities)
Each unique entity becomes a node with type-specific initial feature vector. Heterogeneous

Edges (Interactions)
Each event becomes a directed, timestamped edge connecting source to destination entity.

Node Features
$h_v^{(0)}$ — Type-specific initial embedding. Dimension: d

Edge Features
e_{uv} — Action type encoding + attribute vector. Includes timestamp t_{uv}

Formal Graph Definition

$G_t = (V, E, T, A)$ where V = set of heterogeneous nodes, E = set of directed edges, $T : E \to \mathbb{R}^+$ assigns continuous timestamps, $A : E \to \mathbb{R}^k$ assigns edge feature vectors.

2.2 Continuous-Time Temporal Graph Construction

EDGE TEMPORAL ORDERING

Events are indexed as a chronologically sorted sequence. Each new event $e_i = (u, v, t_i, a_i)$ is appended to the temporal edge list, maintaining the invariant $t_1 \leq t_2 \leq \ldots \leq t_n$.

Sliding Window: A configurable temporal window ΔW retains only recent edges for online inference, preventing unbounded memory growth.

GRAPH STORAGE

In-Memory Graph
CSR format adjacency + timestamp array for fast temporal neighborhc

Persistent Storage
Neo4j / NetworkX serialization for full provenance graph persistence.

LAYER 3 — HETEROGENEOUS CONTINUOUS-TIME TGNN CORE ENGINE

OBJ 1

3.1 Continuous-Time Time Encoding Module

Sinusoidal Time Encoding
Projects scalar timestamps into high-dimensional space preserving temporal proximity relationships.

Time Decay Function
Exponential decay weights recent interactions higher than distant ones.

Sinusoidal Time Encoding

$TE(t) = [sin(\omega_1 t), cos(\omega_1 t), ..., sin(\omega_{d/2} t), cos(\omega_{d/2} t)]$

where $\omega_k = 1 / 10000^{2k/d}$ (positional encoding frequencies)

Temporal Attention Weight

$\alpha_{uv}^{(t)} = exp(-\sigma \cdot |t - t_{uv}|) \cdot softmax(W_a \cdot [h_u \blacktriangle h_v \blacktriangle TE(t_{uv})])$ — Combines temporal recency with learned attention over node states and time embeddings.

3.2 Neighborhood Message Passing (Temporal Attention)

Source Node h_u → Linear Transform W_φ → Temporal Attention α → Weighted Aggregate Σ

Multi-Head Attention: K independent attention heads compute parallel message representations, concatenated and projected: $m_v = ||_{k=1}^{K} W_k \cdot Attn_k(N_t(v))$

Message Computation

$m_v^{(l)}(t) = \Sigma_{u \in N_t(v)} \alpha_{uv}^{(l)}(t) \cdot \varphi(h_u^{(l)}, e_{uv}, TE(t_{uv}))$

$N_t(v)$ = temporal neighborhood of v at time t (all edges $(u,v,t') : t' \leq t$)

3.3 Temporal Aggregation Module

Temporal Attention Pooling
Attention-weighted aggregation over all timestamps in the neighborhood window. Learns which time steps matter most for the current classification.

Sliding Window Aggregation
Aggregates messages within a configurable time window ΔW, enabling efficient online processing without recomputing full history.

RNN/GRU Temporal Encoder
Sequentially processes chronologically ordered messages through a GRU cell, capturing temporal dynamics and event ordering effects.

Node State Update

$h_v^{(l+1)}(t) = GRU(h_v^{(l)}(t), AGG(\{m_v^{(l)}(t_i) : t_i \in [t - \Delta W, t]\}))$ — Updates node embedding by aggregating temporally-weighted messages from neighbors.

3.4 Persistent Entity Memory Module

Entity Memory Bank
Per-node trainable memory matrix $M_v \in \mathbb{R}^{m \times d}$ that persists across timesteps.

Memory Read/Write Gates
Learned gates control which information is read from and written to memory at each event.

Purpose: Captures long-term behavioral patterns for each entity (e.g., "User X typically accesses these files every morning"). Enables detection of deviations from established baselines.

Memory Update Mechanism

$r_v^{(t)} = \sigma(W_r \cdot [h_v^{(l)}(t) \,||\, m_v^{(old)}])$ — Read gate

$w_v^{(t)} = \sigma(W_w \cdot [h_v^{(l)}(t) \,||\, m_v^{(old)}])$ — Write gate

$m_v^{(new)} = r_v \odot m_v^{(old)} + w_v \odot h_v^{(l)}(t)$ — Memory update

3.5 Heterogeneous Type-Aware Processing

Type-Specific Projection
Separate linear projections W_τ for each node/edge type τ, enabling type-aware feature transformation.

Type Embedding
Learnable type embeddings e_τ concatenated to node features, providing structural type inform

Cross-Type Attention
Attention mechanism that modulates message strength based on source-target type pair, capturing cross-type interaction significance.

Type-Aware Message Function

$$\varphi_{\tau(u),\tau(v),\tau(e)}(h_u, e_{uv}, TE(t)) = W_{\tau(v)} \cdot MLP([W_{\tau(u)}h_u \,||\, W_{\tau(e)}e_{uv} \,||\, TE(t_{uv}) \,||\, e_{\tau(u)} \,||\, e_{\tau(v)}])$$

3.6 Classification Head

Final Embedding $h_v^{(L)}(t)$ → **MLP Decoder** → **Softmax / Sigmoid** → **Attack Label $\hat{y}$**

Binary Classification: $\hat{y} = \sigma(W_{out} \cdot h_v^{(L)}(t) + b)$ — Malicious vs. Benign event prediction.

Multi-Class: APT, LotL, Zero-day, Benign — Cross-type attack categorization with hierarchical classification.

Output: Event-level classification with confidence score, plus node-level anomaly scores for graph-wide detection.

LAYER 4 — ONLINE CONCEPT DRIFT ADAPTATION

OBJ 2

4.1 Drift Detection Engine

Statistical Drift Detector
ADWIN (Adaptive Windowing) or Page-Hinkley test monitors prediction error distribution for significant changes.

Feature Distribution Monitor
KL divergence / KS test compares incoming feature distributions against reference window statistics.

Drift Detection Condition

H_0: $D(P_{ref}, P_{new}) \leq \varepsilon$ (null hypothesis: no drift)

H_1: $D(P_{ref}, P_{new}) > \varepsilon$ (drift detected)

When H_1 accepted → trigger adaptation pipeline.

4.2 Incremental Model Adaptation

Drift Signal → **Buffer Recent Labeled Data** → **Selective Replay** → **Fine-Tune TGNN** → **EWC Regularization** → **Updated Model**

ELASTIC WEIGHT CONSOLIDATION (EWC)

Prevents Catastrophic Forgetting: Computes Fisher Information Matrix F on old data. Adds penalty $\lambda \Sigma_i F_i(\theta_i - \theta_i^*)^2$ to loss, constraining important parameters from changing too much.

EXPERIENCE REPLAY BUFFER

Maintains Old Knowledge: Reservoir-sampled buffer of representative events interleaved with new data during fine-tuning. Buffer size B with FI replacement.

&circlearrow; Continuous Feedback Loop: Detection Accuracy Monitor → Drift Detection → Model Update → Accuracy Re-evaluation → (repeat)

LAYER 5 — ATTACK BACKTRACKING & PATH RECONSTRUCTION ENGINE

OBJ 3

5.1 Alert-Triggered Backtracking

Detection Alert $\hat{y}_v(t)$ → Identify Alert Node → Reverse BFS/DFS → Temporal Path Scoring → Attack Chain

Temporal Subgraph Extraction

Extracts the connected temporal subgraph G'_t rooted at the alert node, traversing backward through temporal edges.

Attention-Based Path Ranking

Uses TGNN attention weights α_{uv} to rank edges by contribution to the malicious classification.

Path Significance Score

$$S(path) = \Sigma_{e_i \in path} \alpha_{e_i} \cdot P(malicious|e_i)$$

$$\times\ temporal\_coherence(path)$$

Paths with $S(path) > threshold\ \tau$ are included in the reconstructed att

5.2 Multi-Step Attack Scenario Reconstruction

Initial Compromise Identification

Finds the earliest node in the attack chain — the point of initial entry (e.g., phishing email, exploited vulnerability).

Lateral Movement Tracing

Traces the attacker's movement across the network: which machines were compromised, what credentials were used, what data was accessed.

Impact Assessment

Identifies all affected entities (files, credentials, machines) and the blast radius of the attack for IR prioritization.

Output Format: Chronologically ordered attack timeline with entities, actions, timestamps, and confidence scores for each step in the chain.

Visualization: Interactive temporal graph visualization (pyvis/Plotly) highlighting the attack path in red, normal activity in gray.

LAYER 6 — TEMPORAL EXPLAINABILITY VIA MITRE ATT&CK MAPPING

OBJ 4

6.1 Attention Weight Extraction & Temporal Mapping

TGNN Attention Weights $\{\alpha_{uv}\}$ → Top-K Edge Selection → Temporal Ordering → Behavioral Pattern Extraction

Attention Aggregation: Per-edge attention scores are aggregated per entity pair and time window to identify the most influential interactions contributing to the classification.

Temporal Pattern: Extracts **when** (timestamp progression), **how** (action sequence), and **why** (attention-weighted significance) each event contributed.

6.2 MITRE ATT&CK Framework Mapping

Tactic-Level Mapping
Maps detected behavior sequences to MITRE ATT&CK tactics (Initial Access, Execution, Persistence, Privilege Escalation, Lateral Movement, Exfiltration).

Technique-Level Mapping
Fine-grained mapping to specific techniques (e.g., T1059.001 — PowerShell, T1003 — OS Credential Dumping).

Mapping Algorithm: Rule-based + embedding similarity. Each detected action type is matched against the MITRE ATT&CK technique database using action-type taxonomy and behavioral features.

OUTPUT ANNOTATIONS

Tactic Tag
TA0001 Initial Access

Technique Tag
T1566.001 Spearphishing Attach

6.3 LLM-Generated Natural Language Explanation

Attack Path + ATT&CK Mapping → Structured Prompt Builder → LLM (GPT / Local Model) → Human-Readable Narrative

PROMPT TEMPLATE

"Given the following temporal attack sequence: [chronological events with timestamps, entities, actions, ATT&CK technique IDs, and attention scores], generate a security analyst report explaining what was detected, when each step occurred, how the attacker progressed, and why each action was classified as malicious."

EXPLANATION DIMENSIONS

WHAT
The detected attack type and affected entities.

WHEN
Temporal prog

HOW
Attack techniques used, mapped to ATT&CK.

WHY
Model reasonin

LAYER 7 — ONLINE INFERENCE & DEPLOYMENT

7.1 Real-Time Event Stream Processing

Live Log Stream → Event Parser → Graph Update → TGNN Inference → Alert + Explanation

Latency Target: End-to-end processing ≤ 500ms per event from log ingestion to alert generation.

API Layer: FastAPI REST endpoints for event submission, alert querying, and explanation retrieval.

Dashboard: Streamlit + Plotly + pyvis for real-time temporal graph visualization, alert monitoring, and attack path exploration.

Data Ingestion (OBJ 1) | Graph Construction (OBJ 1) | TGNN Core (OBJ 1) | Concept Drift (OBJ 2) | Attack Backtracking (OBJ 3) | Explainability (OBJ 4) | Ge